

## Part 7: The Distributed Architecture in Practice

# PART 7: THE DISTRIBUTED ARCHITECTURE IN PRACTICE

### Problem Inventory

| #   | Problem                                                                       | Severity | Root Cause                                     |
|-----|-------------------------------------------------------------------------------|----------|------------------------------------------------|
| 7.1 | No replication between any PG nodes                                           | Critical | TLS cert issue deferred, never resolved        |
| 7.2 | Alphabetical customer fragmentation (A-M / N-Z) correlates with 0% of queries | Critical | No query pattern analysis before fragmentation |
| 7.3 | Orders split (Online/Retail) but 78% of queries join both                     | High     | Wrong fragmentation key                        |
| 7.4 | Middleware performs joins in Node.js memory — crashes on large joins          | Critical | No database-level federation                   |
| 7.5 | Order placement has no atomicity across nodes — 340 partial failures/week     | Critical | No distributed transaction protocol            |
| 7.6 | Lahore nodes 6 months stale                                                   | High     | Consequence of 7.1                             |
| 7.7 | NovaPay writes to Node 1 instead of Node 4                                    | Medium   | Temporary network routing fix made permanent   |

- **7.1 No replication between any PG nodes**

- **Problem:** The system has zero fault tolerance; a single-node failure (like the ransomware event) results in total data unavailability.
- **Solution:** Configure PostgreSQL Streaming Replication with standby nodes.
- **Why Chosen:** Streaming replication provides immediate high availability and allows for read-replicas, ensuring data survives disasters without the massive latency overhead of purely synchronous distributed architectures.

- **7.2 Alphabetical customer fragmentation (A-M/N-Z)**

- **Problem:** Data is sharded by surname initial, which correlates with 0% of queries, causing massive cross-node lookups.

- **Solution:** Consolidate the customer data to Node 1 and partition it by hash on customer\_id.
- **Why Chosen:** Horizontal fragmentation must logically align with actual query access patterns; hash partitioning evenly distributes load for the direct ID lookups that dominate the system.
- **7.3 Orders split (Online/Retail)**
  - **Problem:** Orders are physically split by channel across nodes, but 78% of queries join both channels to get a holistic view of the customer.
  - **Solution:** Consolidate orders to Node 1 and partition them by range based on order\_date.
  - **Why Chosen:** Range partitioning naturally supports the time-series nature of order filtering and allows for the easy archiving of older data without cross-node joins.
- **7.4 Middleware performs joins in Node.js memory**
  - **Problem:** The Node.js application pulls massive unjoined datasets from separate databases, causing out-of-memory crashes.
  - **Solution:** Implement a PostgreSQL Foreign Data Wrapper (postgres\_fdw) to link Node 3's inventory directly to Node 1.
  - **Why Chosen:** Pushing heavy join operations down to the database engine utilizes its mathematical optimizations for set operations and prevents middleware memory bottlenecks.

## What Happens to Nodes 2, 3, 4, 5?

| Node            | Current Role          | New Role                                                                                                                              |
|-----------------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Node 1          | Primary OLTP          | Primary OLTP (consolidated)                                                                                                           |
| Node 2          | Customers A-M         | <b>Streaming standby of Node 1</b> (read replicas for load distribution)                                                              |
| Node 3          | Inventory + Logistics | <b>Dedicated inventory node</b> (keep inventory here, it's the one table that benefits from separation — POS queries are store-local) |
| Node 4 (Lahore) | NovaPay (unused)      | <b>Warm standby of Node 1</b> (DR site)                                                                                               |
| Node 5 (Lahore) | Analytics staging     | <b>Analytics staging</b> (fed by logical replication from Node 1)                                                                     |

- **7.5 Order placement has no atomicity across nodes**

- **Problem:** Distributed transactions fail partially, resulting in phantom inventory issues and partial orders.
- **Solution:** Implement the Saga orchestration pattern driven by a reliable Outbox table.
- **Why Chosen:** The Outbox pattern guarantees at-least-once delivery and eventual consistency across nodes without introducing the severe performance locking of Two-Phase Commit (2PC).

- **7.6 Lahore nodes 6 months stale**

- **Problem:** Nodes 4 and 5 in Lahore are completely outdated due to broken replication and TLS certificate issues.
- **Solution:** Re-establish logical replication and convert the Lahore nodes to proper streaming/warm standbys.
- **Why Chosen:** Restores disaster recovery capabilities and ensures the analytics node has fresh data without taxing the primary OLTP database.

- **7.7 NovaPay writes to Node 1 instead of Node 4**

- **Problem:** A temporary network routing fix was made permanent, forcing NovaPay transactions to the main OLTP node and overloading it.

- **Solution:** Reroute NovaPay traffic back to its dedicated Node 4 and sync via replication.
- **Why Chosen:** Decouples NovaPay processing from the primary node, isolating workloads and preventing resource contention.